

Résumé du projet : Tennis — Gestion de joueurs et de tournois

Titre professionnel Développeur Web et Web Mobile — Salah Belhassan — La Plateforme, session 2026

Introduction : Instant Tennis est une API REST sécurisée et développée en Spring Boot, permettant de gérer des joueurs, des tournois et les inscriptions des joueurs aux tournois. L'application repose sur une persistance relationnelle PostgreSQL versionnée par Flyway, une architecture en couches (controllers → services → repositories), une sécurité par sessions Spring Security avec contrôle d'accès par rôle, et un déploiement conteneurisé avec Docker.

ACTIVITÉ TYPE 2 : développer la partie back-end d'une application web ou web mobile sécurisée

CP 5 — Mettre en place une base de données relationnelle

- **Modélisation :** base PostgreSQL 17 structurée autour des tables player, tournament, dyma_user et dyma_role, complétées par deux tables de jointure (player_tournament, dyma_user_role) portant les relations plusieurs-à-plusieurs entre joueurs/tournois et utilisateurs/rôles.
- **Migrations versionnées :** dix scripts Flyway (V001 à V010) organisés en dossiers common / dev / prod / test, chacun avec son propre jeu de données (joueurs et comptes de démonstration en dev, données réelles injectées en prod, jeux dédiés aux tests d'intégration).
- **Intégrité portée par la base :** contraintes d'unicité sur l'identifiant UUID et le nom de chaque tournoi, contrainte d'unicité composite (prénom, nom, date de naissance) sur les joueurs, clés étrangères sur les tables de jointure — un contrôle applicatif contourné serait rattrapé par la base.
- **Déploiement :** conteneur Docker PostgreSQL (image officielle) piloté par docker-compose avec volume de persistance dédié, configuration différenciée dev/prod via les profils Spring et des propriétés externalisées (identifiants injectés en production, jamais commités).

CP 6 — Développer des composants d'accès aux données SQL et NoSQL

- **Accès SQL :** Spring Data JPA (Hibernate) avec un repository par entité (PlayerRepository, TournamentRepository, UserRepository), méthodes dérivées (findOneByIdentifiant, findOneByNameIgnoreCase, findOneWithRolesByLoginIgnoreCase) traduites en requêtes préparées — protection native contre l'injection SQL.
- **Relations objet-relationnel :** associations bidirectionnelles @ManyToMany entre joueurs et tournois (table player_tournament) et entre utilisateurs et rôles (table dyma_user_role), avec un chargement maîtrisé pour éviter les accès en cascade non désirés.
- **Persistance de test isolée :** base H2 embarquée dédiée aux tests d'intégration et de bout en bout, rejouée par Flyway (clean puis migrate) avant chaque scénario pour garantir l'indépendance des cas de test.
- **Périmètre NoSQL :** ce projet ne mobilise pas de base documentaire ; l'ensemble de la persistance métier reste relationnel (PostgreSQL), ce qui a permis d'approfondir Spring Data JPA plutôt que d'introduire un second modèle de données.

CP 7 — Développer des composants métier côté serveur

- **Architecture en couches :** controllers → services → repositories pour chacune des ressources métier (joueurs, tournois, inscriptions), avec des mappers dédiés (PlayerMapper, TournamentMapper) qui isolent les entités JPA des objets exposés par l'API.
- **Règles métier d'inscription :** RegistrationService vérifie systématiquement côté serveur l'existence du tournoi et du joueur, le nombre de places restantes et l'absence de double inscription avant toute écriture — aucune confiance n'est accordée à ce qu'enverrait un client.
- **Calcul de classement :** RankingCalculator recalcule et réattribue le rang de chaque joueur à partir de ses points, triés par ordre décroissant.

- **Gestion d'erreurs typée** : sept exceptions métier dédiées (PlayerNotFoundException, PlayerAlreadyExistsException, TournamentNotFoundException, TournamentAlreadyExistsException, TournamentRegistrationException, PlayerDataRetrievalException, TournamentDataRetrievalException) interceptées par des `@RestControllerAdvice` qui les traduisent en codes HTTP appropriés (400, 404, 500) et en erreurs de validation par champ.
- **Authentification et autorisation** : Spring Security avec mots de passe hachés en BCrypt, chargement de l'utilisateur et de ses rôles via `DynaUserService`, session applicative gérée côté serveur (login/logout dans `AccountController`) et contrôle d'accès déclaratif par rôle et par verbe HTTP (lecture réservée à `ROLE_USER`, écriture et administration réservées à `ROLE_ADMIN`, endpoints Actuator réservés à `ROLE_ADMIN`) ; en-têtes de sécurité activés (Content-Security-Policy, X-Frame-Options à deny, Permissions-Policy).
- **Tests** : JUnit 5 et Mockito pour les tests unitaires de services (dépendances simulées), complétés par des tests d'intégration et des tests de bout en bout via `TestRestTemplate` qui rejouent les migrations Flyway avant chaque cas, pour un total de quinze classes de test.
- **Documentation d'API** : endpoints décrits avec springdoc-openapi (Swagger UI), annotations `@Operation` et `@ApiResponse`s précisant les codes de retour possibles pour chaque route.

CP 8 — Documenter le déploiement d'une application dynamique

- **Image applicative** : Dockerfile multi-étapes (construction Maven avec le profil prod, puis image d'exécution allégée eclipse-temurin JRE), exécutée sous un utilisateur dédié non-root pour limiter les privilèges du conteneur.
- **Infrastructure locale** : service PostgreSQL et pile de supervision des logs (Graylog, Elasticsearch, MongoDB support de Graylog) décrits en docker-compose, avec volumes de persistance dédiés.
- **Configuration par profils** : propriétés Spring externalisées et distinctes pour dev (base et scripts de démonstration) et prod (identifiants injectés, jamais commités, journalisation restreinte au niveau WARN).
- **Observabilité** : Spring Boot Actuator (health, metrics) protégé par rôle, logs structurés au format GELF envoyés à Graylog via logback-gelf, traçage des requêtes SQL en développement grâce à datasource-proxy.
- **Documentation et reproductibilité** : README présentant le projet, Maven Wrapper garantissant un build identique sur tout poste, profils Maven dev/prod alignés sur les profils Spring pour un packaging cohérent avant conteneurisation.